Algoritmo e Laboratório de Programação II

Aula 7 – Ponteiros - parte 3 (Nível Avançado)

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Ponteiros Para Funções

- Apresentaremos um tipo de ponteiro especial: um ponteiro que aponta para uma função, ou seja, uma variável que irá conter o endereço de uma função.
- A função poderá ser executada por meio do ponteiro.
- O nosso primeiro exemplo mostra um ponteiro para a função doisbeep().
- Eis a listagem:

Ponteiros Para Funções

Prog18.c

```c
/* Prog18.c */
/* Mostra o uso de ponteiro para função */
#include <stdio.h>
#include <stdlib.h>

void doisbeep(void); /* protótipo */

int main()
{
    void (*pf)(void);/* ponteiro p/fção void que recebe void */

    pf = doisbeep; /* nome da função sem os parênteses */

    (*pf)();/* chama a função */

    system("PAUSE");
    return 0;
}

/* doisbeep() */
/* toca o alto-falante duas vezes */
void doisbeep(void)
{
    unsigned i;
    printf("\a");
    for(i=0; i < 800000 ; i++); /* dar um tempo */
    printf("\a");
}
```

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Declarando o Ponteiro Para Função

- A função main() começa declarando pf como um ponteiro para uma função void.
- É claro que o tipo void é uma das possibilidades.
- Se a função a ser apontada é do tipo float, por exemplo, o ponteiro para ela deve ser declarado como tal.

```
void (*pf)(void);
```

Declarando o Ponteiro Para Função

- Observe os parênteses envolvendo *pf.
- Esses parênteses são realmente necessários, pois, se omitidos

```
void *pf(void);
```

- Estaríamos declarando pf como sendo uma função que retorna um ponteiro void;
- Em outras palavras, estaríamos escrevendo o protótipo da função pf.

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Endereços de Funções

- O nome de uma função desacompanhado de parenteses é o seu endereço.
- A instrução: `pf = doisbeep;`
- Atribui o endereço da função doisbeep() a pf.
- Observe que não colocamos parenteses junto ao nome da função.
- Se eles estivessem presentes, como em

```
pf = doisbeep();
```

- Estaríamos atribuindo a pf o valor de retorno da função, e não o seu endereço.

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Executando a Função por Meio do Ponteiro

- Da mesma forma que podemos substituir o nome de uma variável usando um ponteiro acompanhado do operador indireto *, podemos substituir o nome da função usando o mesmo mecanismo.
- A instrução `(*pf)();`
- É equivalente a `doisbeep();`
- E indica uma chamada à função doisbeep().

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Operações Ilegais com Ponteiros para Funções

- A aritmética com ponteiros para funções não é definida em C.
- Por exemplo, você não pode incrementar ou decrementar ponteiros para funções.

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Ponteiros para Funções como Argumentos

- O exemplo a seguir cria um ponteiro para armazenar o endereço da função de biblioteca gets().
- Essa função tem o seguinte protótipo:

```
char *gets(char *);
```

- Definido no arquivo stdio.h.
- A função retorna um ponteiro para a cadeia de caracteres lida do teclado e armazenada no endereço recebido por ela como argumento.

Ponteiros para Funções como Argumentos

Prog19.c

```c
/* Prog19.c */
/* Mostra o uso de ponteiro como argumento de função */
#include <stdio.h>
#include <stdlib.h>

void func( char * (*)(char *));

int main()
{
    char * (*p)(char *);
    p = gets;
    func(p);

    system("PAUSE");
    return 0;
}

void func(char * (*p)(char *))
{
    char nome[80];

    printf("Digite seu nome: ");

    (*p)(nome); /* chama a função gets()*/

    printf("Seu nome é: %s\n", nome);
}
```

Ponteiros para Funções como Argumentos

- A declaração do ponteiro é a seguinte:

```
char * (*p)(char *);
```

- Essa instrução indica que p é um ponteiro para uma função do tipo char* e recebe um char* como argumento.
- A função main() envia o ponteiro p como argumento para a função func().

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Matrizes de Ponteiros para Funções

- Os ponteiros oferecem uma maneira eficiente de executar uma função entre uma série delas, com base em alguma escolha dependente de parâmetros conhecidos somente em tempo de execução.
- Por exemplo, suponhamos que você queira escrever um programa em que é apresentado um “menu” de opções ao usuário, e, para cada escolha, o programa deve executar uma chamada a uma função particular.

Matrizes de Ponteiros para Funções

- Em vez de utilizar estruturas tradicionais de programação como switch ou if-else, ou qualquer outra estrutura de controle para decidir qual função deve ser chamada, você simplesmente cria uma matriz de ponteiros para funções e executa a função correta por meio de seu ponteiro.
- Eis um esqueleto do programa:

Matrizes de Ponteiros para Funções

Prog20.c

```c
/* Prog20.c */
/* Mostra uma matriz de ponteiros para função */
#include <stdio.h>
#include <stdlib.h>

const true=1;

void func0(void), func1(void), func2(void);/* Protótipos */

int main()
{
    void (*ptrf[3])(void);/* Matriz de ponteiros para funções */

    ptrf[0] = func0;
    ptrf[1] = func1;
    ptrf[2] = func2;

    do
    {
        int i;
        printf("0 - ABRIR\n");
        printf("1 - FECHAR\n");
        printf("2 - SALVAR\n");
        printf("\nEscolha um item: ");
        scanf("%d", &i);
        if(i < 0 || i > 2) break;
```

Matrizes de Ponteiros para Funções

```
28              (*ptrf[i])(); /* Chama função */
29          }
30          while(true);
31          system("PAUSE");
32          return 0;
33      }
34
35      void func0()
36      {
37          printf("\n*** Estou em func0() ***\n");
38      }
39
40      void func1()
41      {
42          printf("\n*** Estou em func1() ***\n");
43      }
44
45      void func2()
46      {
47          printf("\n*** Estou em func2() ***\n");
48      }
```

Matrizes de Ponteiros para Funções

- A instrução:

```
void (*ptrf[3])(void);
```

- Declara uma matriz de três ponteiros para funções void.
- O laço do-while em main() chama uma das três chama uma das três funções, dependendo da escolha do usuário.

Matrizes de Ponteiros para Funções

- A instrução:

```
void (*ptrf[3])(void);
```

- Declara uma matriz de três ponteiros para funções void.
- O laço do-while em main() chama uma das três chama uma das três funções, dependendo da escolha do usuário.

Matrizes de Ponteiros para Funções

- O programa termina se um número menos que zero ou maior que dois for digitado.
- Matrizes de ponteiros para funções fornecem um mecanismo alternativo de mudar o controle do programa sem utilizar estruturas de controle convencionais.

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Inicializando uma matriz de ponteiros para Funções

- O programa anterior poderia ter inicializado a matriz ptrf na mesma instrução de sua declaração.
- Eis a modificação:

Inicializando uma matriz de ponteiros para Funções

Prog21.c

```
1  /* Prog21.c */
2  /* Mostra uma matriz de ponteiros para função */
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  const true=1;
7
8  void func0(void), func1(void), func2(void);/* Protótipos */
9
10 int main()
11 {
12     void (*ptrf[3])(void)= { func0,func1,func2}; /*Matriz inicializada*/
13
14     do
15     {
16         int i;
17         printf("0 - ABRIR\n");
18         printf("1 - FECHAR\n");
19         printf("2 - SALVAR\n");
20         printf("\nEscolha um item: ");
21         scanf("%d", &i);
22         if(i < 0 || i > 2) break;
```

Inicializando uma matriz de ponteiros para Funções

```
24          (*ptrf[i])(); /* Chama função */
25      }
26      while(true);
27      system("PAUSE");
28      return 0;
29  }
30
31  void func0()
32  {
33      printf("\n*** Estou em func0() ***\n");
34  }
35
36  void func1()
37  {
38      printf("\n*** Estou em func1() ***\n");
39  }
40
41  void func2()
42  {
43      printf("\n*** Estou em func2() ***\n");
44  }
```

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Usando typedef para Declarar um Ponteiros para Funções

- É comum definir um nome para um tipo de dado que seja ponteiro para função.
- Fazemos isso por meio de typedef.

Usando typedef para Declarar um Ponteiros para Funções

Prog22.c

```
1  /* Prog22.c */
2  /* Mostra uma matriz de ponteiros para função */
3  #include <stdio.h>
4  #include <stdlib.h>
5  void func0(void), func1(void), func2(void);/* Protótipos */
6
7  const true=1;
8
9  typedef void (*PFunc)(void); /* O tipo PFunc é ponteiro p/fção void */
10
11 int main()
12 {
13     PFunc ptrf[3] = { func0, func1, func2};/* Matriz inicializada */
14
15     do
16     {
17         int i;
18         printf("0 - ABRIR\n");
19         printf("1 - FECHAR\n");
20         printf("2 - SALVAR\n");
21         printf("\nEscolha um item: ");
22         scanf("%d", &i);
23         if(i < 0 || i > 2) break;
```

Usando typedef para Declarar um Ponteiros para Funções

```
25              (*ptrf[i])(); /* Chama função */
26          }
27          while(true);
28          system("PAUSE");
29          return 0;
30      }
31
32      void func0()
33      {
34          printf("\n*** Estou em func0() ***\n");
35      }
36
37      void func1()
38      {
39          printf("\n*** Estou em func1() ***\n");
40      }
41
42      void func2()
43      {
44          printf("\n*** Estou em func2() ***\n");
45      }
```

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Ponteiros void

- Antes de prosseguir a explanação de ponteiros, vamos apresentar um ponteiro peculiar.
- Quando queremos atribuir um endereço a um ponteiro, este deve ser do mesmo tipo daquele.
- Por exemplo, não podemos atribuir um endereço de uma variável int a um ponteiro float.

Ponteiros void

- Entretanto, há uma exceção.
- Há um tipo de ponteiro de propósito geral que pode apontar para qualquer tipo de dado.
- Esse ponteiro é do tipo void, podendo ser declarado por meio da seguinte instrução:

```
void *p; /* p aponta para qualquer tipo de dado */
```

Ponteiros void

- Ponteiros do tipo void são usados em situações em que seja necessário que uma função receba ou retorne um ponteiro genérico e opere independentemente do tipo de dado apontado.
- Observe que o conceito de ponteiros void não tem absolutamente nada a ver com o tipo void para funções.
- Qualquer endereço pode ser atribuído a um ponteiro void:

Ponteiros void

Prog23.c

```c
/* Prog23.c */
/* Mostra ponteiros void */
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int i=5;
    float f=3.2;

    void *pv;    /* Ponteiro genérico */
    pv = &i;     /* Endereço de um int */

    /* não podemos usar o operador indireto com ponteiros void */
    printf("%d\n", *pv); /* ERRO de compilação */

    pv = &f;     /* Endereço de um float */

    /* não podemos usar o operador indireto com ponteiros void */
    printf("%f\n", *pv); /* ERRO de compilação */

    system("PAUSE");
    return 0;
}
```

Ponteiros void

- O conteúdo da variável apontada por um ponteiro void não pode ser acessado por meio desse ponteiro.
- É necessário criar outro ponteiro e fazer a conversão de tipo na atribuição:

Ponteiros void

Prog24.c

```c
/* Prog24.c */
/* Mostra ponteiros void */
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int i=5, *pi;
    float f=3.2, *pf;

    void *pv;    /* Ponteiro genérico */
    pv = &i;     /* Endereço de um int */

    pi = (int *)pv; /* Convertendo o tipo do ponteiro */
    printf("%d\n", *pi); /* Correto */

    pv = &f;     /* Endereço de um float */

    pf = (float *) pv; /* Convertendo o tipo do ponteiro */
    printf("%f\n", *pf);/* Correto */

    system("PAUSE");
    return 0;
}
```

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

A Função qsort()

- A função de biblioteca-padrão qsort() executa a ordenação de uma matriz por meio da implementação do algoritmo “quick-sort”.

A Função qsort()

- A função é do tipo void e recebe quatro argumentos:
  - O primeiro é o endereço da matriz a ser ordenada, o segundo é um inteiro com o número de elementos da matriz, o terceiro é o tamanho em bytes de um elemento da matriz e o último é o endereço de uma função com o seguinte protótipo:

```
int compara(const void *, const void *);
```

A Função qsort()

- A função compara() deve ser escrita pelo usuário de qsort() e compara dois elementos da matriz.
- Retorna um valor que informa o resultado da comparação:

Valor retornado pela função compara()	Descrição
< 0	elem1 menor que elem2
0	elem1 igual ao elem2
> 0	elem1 maior que elem2

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Ordenando Números Inteiros

- Nosso primeiro exemplo mostra a ordenação de uma matriz de inteiros:

Prog25.c

```
1   /* Prog25.c */
2   /* Algoritmo qsort com inteiros */
3   #include <stdio.h>
4   #include <stdlib.h>
5
6   int compara(const void *, const void *);
7
8   int main()
9   {
10      unsigned int tamanho, i;
11
12      int tab[]= { 234, 760, 162, 890, -23, 914, 567, 888, 398, -45};
13
14      tamanho = sizeof(tab)/sizeof(int);
```

Ordenando Números Inteiros

```
16      qsort(tab, tamanho, sizeof(int), compara);
17
18      for(i=0; i< 8; i++) printf("%d\n", tab[i]);
19      system("pause");
20      return 0;
21  }
22
23  /* qsort - função para comparar inteiros */
24  int compara(const void *a, const void *b)
25  {
26      const int *pa = (const int *)a; /* modifica o tipo do ponteiro */
27      const int *pb = (const int *)b; /* modifica o tipo do ponteiro */
28      return *pa  - *pb;  /* Retorna negativo se a<b e positivo se a > b  */
29  }
```

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

O Algoritmo de Procura Binária

- O algoritmo de procura binária é, de modo geral, o mais eficiente (demora menos tempo para encontrar o valor), mas requer que a lista de itens esteja ordenada.
- Ele primeiramente seleciona um valor no meio da lista e o compara ao valor que está sendo procurado.
- Se o valor procurado for maior que o valor selecionado, repete-se o processo para a metade da lista posterior ao valor selecionado.

O Algoritmo de Procura Binária

- Se o valor procurado for menor que o valor selecionado, repete-se o processo da lista anterior ao valor selecionado.
- O processo é repetido até um valor ser encontrado, ou até a metade em que a procura deverá ser feita seja esvaziada (neste caso o valor não está na lista).

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Procura Binária com Números Inteiros

- O próximo exemplo implementa o algoritmo de procura binária com inteiros.

Prog26.c

```c
/* Prog26.c */
/* Algoritmo qsort e binarySearch com inteiros */
#include <stdio.h>
#include <stdlib.h>

/* Função procura binária para inteiros
 * procura entre MatrizOrdenada[inicio]..MatrizOrdenada[fim] pela chave.
 * retorna o índice do elemento encontrado ou -1 se não foi encontrado */
int binarySearchInt(int MatrizOrdenada[], int inicio, int fim, int chave)
{
    while (inicio <= fim)
    {
        int meio = (inicio + fim) / 2;  /* divide ao meio */
        if (chave > MatrizOrdenada[meio])
            inicio = meio + 1;  /* repete a procura a partir do meio */
        else if (chave < MatrizOrdenada[meio])
            fim = meio - 1; /* repete a procura até o meio */
        else
            return meio;     /* encontrado, retorna posição */
    }
    return -1; /* não foi encontrado */
}
```

Procura Binária com Números Inteiros

```
24  /* qsort - função para comparar inteiros */
25  int compara(const void *a, const void *b)
26  {
27      const int *pa = (const int *)a; /* modifica o tipo do ponteiro */
28      const int *pb = (const int *)b; /* modifica o tipo do ponteiro */
29      return *pa  - *pb;  /* Retorna negativo se a<b e positivo se a > b  */
30  }
31
32  int main()
33  {
34      unsigned int tamanho, i, procura;
35
36      int tab[]= { 234, 760, 162, 890, -23, 914, 567, 888, 398, -45};
37      printf("\nMatriz Original\n");
38      for(i=0; i< 8; i++) printf("%d\n", tab[i]);
39      tamanho = sizeof(tab)/sizeof(int);
40
41      qsort(tab, tamanho, sizeof(int), compara);
42      printf("\nMatriz Ordenada\n");
43      for(i=0; i< 8; i++) printf("%d\n", tab[i]);
44
45      procura = binarySearchInt(tab, 0, tamanho - 1, 567);
46      printf("\n\nIndice de 567 = %d\n",procura);
47
48      system("pause");
49      return 0;
50  }
```

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Procura Binária com c-string

Prog27.c

```c
/* Prog27.c */
/* Algoritmo qsort e binarySearch com C-string */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Função procura binária para C-string
 * procura entre MatrizOrdenada[inicio]..MatrizOrdenada[fim] pela chave.
 * retorna o índice do elemento encontrado ou -1 se não foi encontrado */
int binarySearchStr(char *MatrizOrdenada[],int inicio,int fim,char*chave)
{
    while (inicio <= fim)
    {
        int meio = (inicio + fim) / 2;  /* divide ao meio */
        int cmp = strcmp(chave, MatrizOrdenada[meio]);

        if (cmp > 0)
            inicio = meio + 1; /* repete a procura a partir do meio */
        else if (cmp < 0)
            fim = meio - 1;       /* repete a procura até o meio */
        else
            return meio;          /* encontrado, retorna posição */
    }
    return -1;              /* não foi encontrado */
}
```

Procura Binária com c-string

```
27  /* qsort - função para comparar C-string (matriz char) */
28  int cstring_cmp(const void *a, const void *b)
29  {
30      const char **pa = (const char **)a;
31      const char **pb = (const char **)b;
32      return strcmp(*pa, *pb);
33      /* strcmp -1 se a < b, 1 se a>b e 0 se a==b*/
34  }
35
36  int main()
37  {
38      unsigned int tamanho, i, procura;
39      char *strings[] = { "Zuleima", "Andre", "Carolina",
40                          "Beto", "Fabio", "Denise"
41                        };
42      tamanho = sizeof(strings) / sizeof(char *);
43      puts("Matriz Original");
44      for(i=0; i< tamanho; i++) printf("%s\n", strings[i]);
45
46      qsort(strings, tamanho, sizeof(char *), cstring_cmp);
47
48      puts("\nMatriz Ordenada");
49      for(i=0; i< tamanho; i++) printf("%s\n", strings[i]);
50
51      procura = binarySearchStr(strings, 0, 5, "Carolina");
52      printf("\nIndice de Carolina = %d\n",procura);
53
54      system("pause");
55      return 0;
56  }
```

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Ponteiro para Estruturas

- Existem diversos motivos para se usar ponteiros para estruturas.
- Por exemplo, se você quiser usar a função qsort() para ordenar os dados de uma matriz de estruturas, deverá escrever a função de comparação que recebe ponteiros.

Ponteiro para Estruturas

Prog28.c

```c
/* Prog28.c */
/* Algoritmo qsort e binarySearch com estruturas */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* Exemplo com estrutura */
struct est_pop
{
    char estado[16];
    int pop; /* população */
};

/* qsort - função para comparar struct por população (membro int) */
int struct_cmp_por_pop(const void *a, const void *b)
{
    struct est_pop *pa = (struct est_pop *)a;
    struct est_pop *pb = (struct est_pop *)b;
    return pa->pop - pb->pop;    /* Retorna neg. se a<b e pos. se a>b */
}

/* qsort - função para comparar struct por estado (membro C-string) */
int struct_cmp_por_estado(const void *a, const void *b)
{
    struct est_pop *pa = (struct est_pop *)a;
    struct est_pop *pb = (struct est_pop *)b;
    return strcmp(pa->estado, pb->estado);
}
```

Ponteiro para Estruturas

```
33      { {"Sergipe   ", 1968}, {"Bahia     ", 13815},
34          {"Piauí     ", 3007}, {"Acre      ", 670},
35          {"Rondônia  ",1535 }, {"Tocantins ", 1306 }
36      };
37
38      tamanho = sizeof(structs) / sizeof(struct est_pop);
39      /* imprime matriz de estruturas original */
40      puts("Estrutura Original");
41      for(i=0; i<tamanho; i++)
42          printf("[ estado: %s \t população: %6d000 ]\n",
43                 structs[i].estado, structs[i].pop);
44      puts("================================================");
45      /* ordena usando a função qsort */
46      qsort(structs, tamanho, sizeof(struct est_pop),
47            struct_cmp_por_pop);
48      /* imprime matriz de estruturas ordenada */
49      puts("Estrutura ordenada por população");
50      for(i=0; i<tamanho; i++)
51          printf("[ estado: %s \t população: %6d000 ]\n",
52                 structs[i].estado, structs[i].pop);
53      puts("================================================");
54      puts("Estrutura ordenada por estado");
55      /* reordena usando a função qsort */
56      qsort(structs, tamanho, sizeof(struct est_pop),
57            struct_cmp_por_estado);
58      /* imprime matriz de estruturas ordenada */
59      for(i=0; i<tamanho; i++)
60          printf("[ estado: %s \t população: %6d000 ]\n",
61                 structs[i].estado, structs[i].pop);
62      puts("================================================");
63      system("pause");
64      return 0;
65  }
```

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Declarando um Ponteiro para Estrutura

- A novidade são as duas funções de comparação:

```
/* qsort - função para comparar struct por população (membro int) */
int struct_cmp_por_pop(const void *a, const void *b)
{
    struct est_pop *pa = (struct est_pop *)a;
    struct est_pop *pb = (struct est_pop *)b;
    return pa->pop - pb->pop;   /* Retorna neg. se a<b e pos. se a>b */
}
/* qsort - função para comparar struct por estado (membro C-string) */
int struct_cmp_por_estado(const void *a, const void *b)
{
    struct est_pop *pa = (struct est_pop *)a;
    struct est_pop *pb = (struct est_pop *)b;
    return strcmp(pa->estado, pb->estado);
}
```

- Na instrução:

```
struct est_pop *pa = (struct est_pop *)a;
```

Declarando um Ponteiro para Estrutura

- Declaramos o ponteiro para estrutura pa.
- Primeiro a palavra struct seguida da etiqueta est_pop, então o operador indireto * seguido do nome do ponteiro.
- A sintaxe é a mesma de qualquer outra declaração de ponteiro que já vimos.

Roteiro

- Ponteiro para funções
- Declarando o ponteiro para função
- Endereços de funções
- Executando a função por meio do ponteiro
- Operações ilegais com ponteiros para funções
- Ponteiros para funções como argumentos
- Matrizes de ponteiros para funções
- Inicializando uma matriz de ponteiros para funções
- Usando typedef para declarar um ponteiro para função
- Ponteiros void
- A função qsort()
- Ordenando números inteiros
- O algoritmo de procura binária
- Procura binária com números inteiros
- Procura binária com c-string
- Ponteiros para estruturas
- Declarando um ponteiro para estrutura
- Acessando membros por meio de ponteiros

Acessando Membros Por Meio de Ponteiros

- Você já aprendeu que, se o nome de uma estrutura for conhecido, podemos acessar seus membros usando o operador ponto (.).
- Será que uma construção análoga, usando um ponteiro em vez do nome da variável, poderia ser escrita?
- Ou seja, seria possível escrever a construção seguinte?

```
pa.estado /*ERRO */
```

Acessando Membros Por Meio de Ponteiros

- A resposta é não, pois pa não é uma variável estrutura e sim um ponteiro para uma variável estrutura;
- Além disso, o operador ponto (.) opera somente sobre o nome de uma estrutura.
- C oferece dois métodos para resolver esse problema:
  - o primeiro, menos elegante, é obter o nome da variável apontada por pa por meio do operador indireto (*): `(*pa).estado /* OK */`

Acessando Membros Por Meio de Ponteiros

- Entretanto, essa expressão é de visualização complexa por causa dos parênteses.
- Os parênteses são necessários, pois o operador (.) tem precedência sobre o operador (*).
  - o segundo método, de uso mais comum, é por meio do operador de acesso a membros (->) que consiste no sinal de “menos” (-) seguido do sinal de “maior” (>). Esse operador opera sobre o endereço de uma variável estrutura e não sobre seu nome.

```
pa->estado /* Mais Usado */
```

Acessando Membros Por Meio de Ponteiros

- Um ponteiro para uma estrutura, seguido pelo operador (->) e pelo nome de um membro, trabalha da mesma maneira que o nome da estrutura seguido pelo operador (.) e pelo nome do membro.
- O operador de acesso a membros (->) conecta o endereço de uma estrutura a um membro dela; o operador ponto (.) conecta o nome da estrutura a um membro dela.

Referências

- MIZRAHI, Victorine V. **Treinamento em Linguagem C**, São Paulo: Pearson Prentice Hall, 2008;